IMSET
INSTITUT MAGHRÉBIN DES SCIENCES ÉCONOMIQUES ET DE TECHNOLOGIE
HONORIS UNITED UNIVERSITIES

ACADÉMIE D'ART DE CARTHAGE
HONORIS UNITED UNIVERSITIES

Intitulé du cours: Programmation Java

Chapitre 2 :
Concept de Classe et Objet

Filière: 2ème année - BTS-Développement sur Internet

Enseignante: Abid Arij

Année de formation 2024/2025

Plan

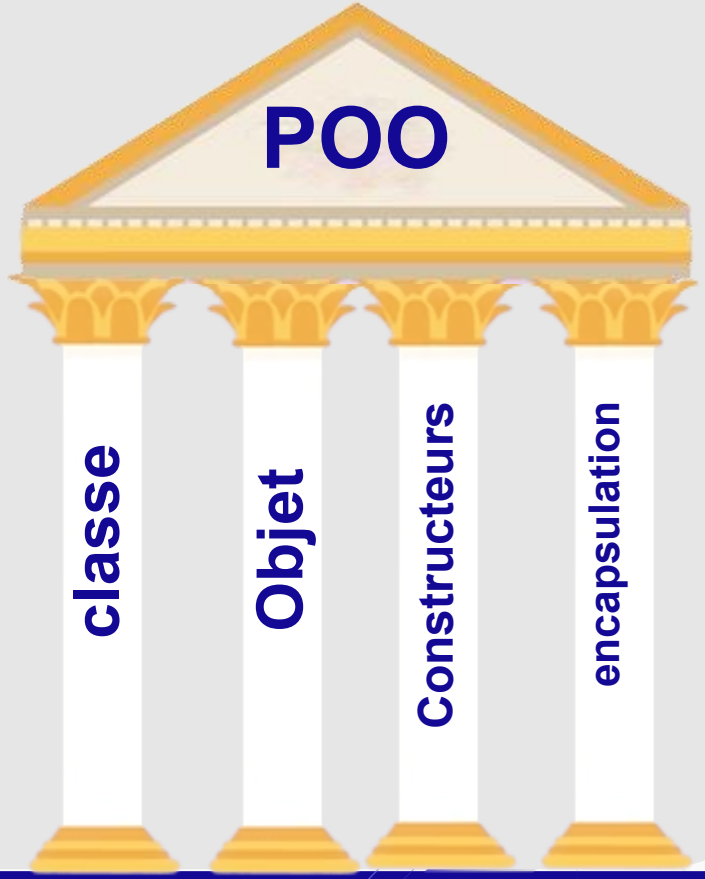
Objectifs du module
Introduction
Définition d’une classe :
–Syntaxe générale
–Exemple : La classe Cercle
Exécution d’un programme java
Les modificateurs d’accessibilité
Les getters et setters
Les constructeurs
Les objets :
–Instanciation
–Accès aux attributs et méthodes
Surcharge des méthodes
Attributs statiques
Les packages

Objectifs du module

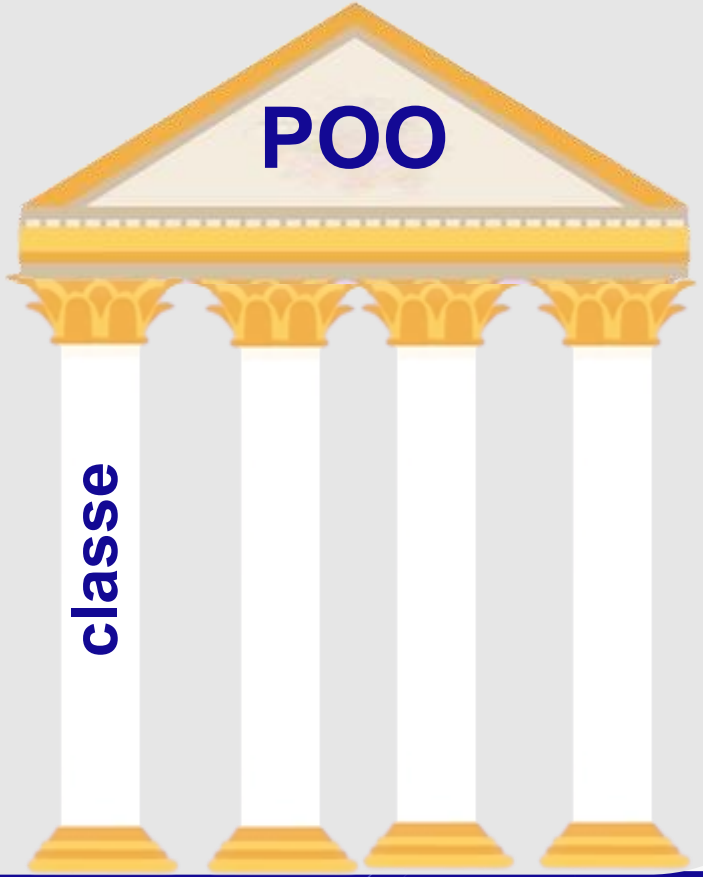
À la fin du module, l'apprenant doit être capable de :

- Comprendre la notion de classe en Java et sa syntaxe de base.
- Créer et manipuler des objets en Java.
- Utiliser les modificateurs d’accessibilité pour contrôler la visibilité des attributs et méthodes.
- Implémenter des **getters** et **setters** pour encapsuler les données.
- Construire des classes avec différents types de **constructeurs**. Instancier des objets et accéder à leurs attributs et méthodes.
- Comprendre et appliquer la surcharge des méthodes. Gérer des attributs **statiques** dans une classe.
- Organiser le code en utilisant des **packages**.
- Exécuter et tester un programme Java.

Introduction



Définition d’une classe



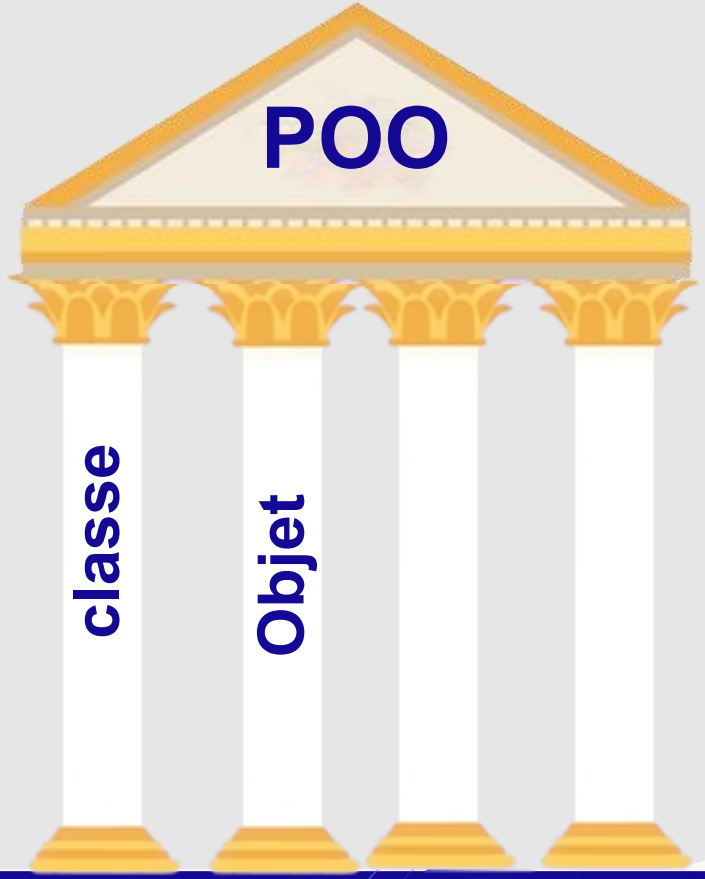
Définition d'une classe

Une classe regroupe :

- **un ensemble de données** (appelées **attributs**)
- **un ensemble de méthodes** de traitement de ces données.

•On parle d'**encapsulation** pour désigner le regroupement des données et des méthodes dans une classe.

Définition d’un objet



Définition d’ Un objet

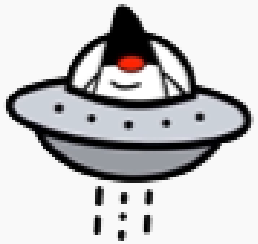
Propriétés

- Portes
- Volant
- Couleur
- Coffre
- Moteur
- Roues
- Phare
- Carrosserie

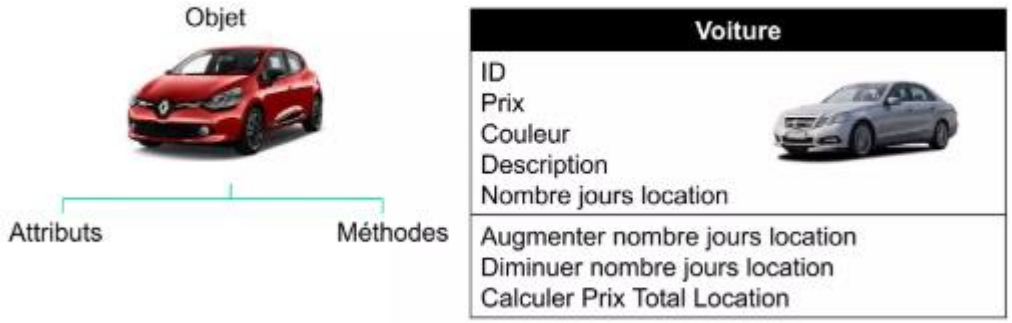
Comportement

- Démarrer le moteur
- Tourner à droite
- Accélérer
- Ouvrir le coffre
- Tourner à gauche
- Arrêter le moteur
- Freiner
- Allumer les phares

Définition d’un objet



•Un objet est une variable comme les autres, en conséquence Il faut qu’il soit déclaré avec son type.
•Le type d’un objet est un type composé (par rapport aux types primitifs simple entier, réel, …) qu’on appelle une **classe**.



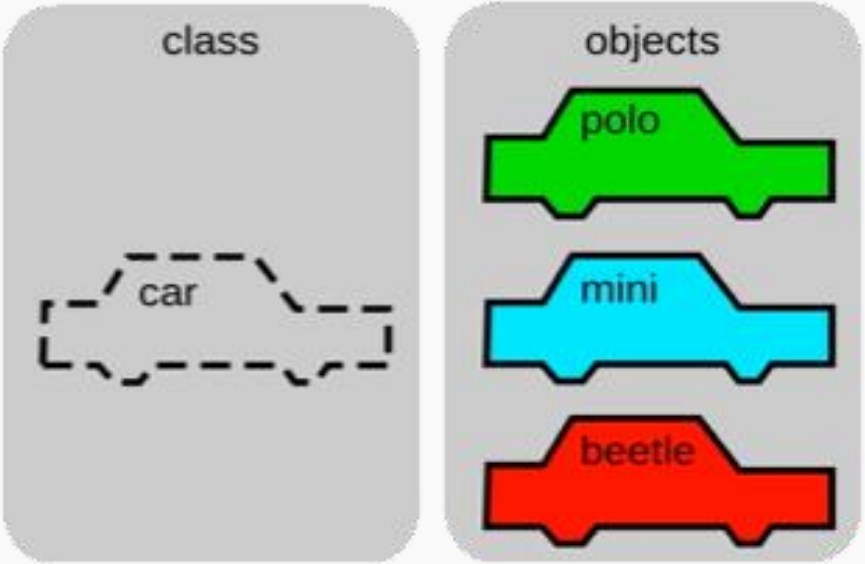
Classe et objet

- **Classe :** Structure définissant un objet.

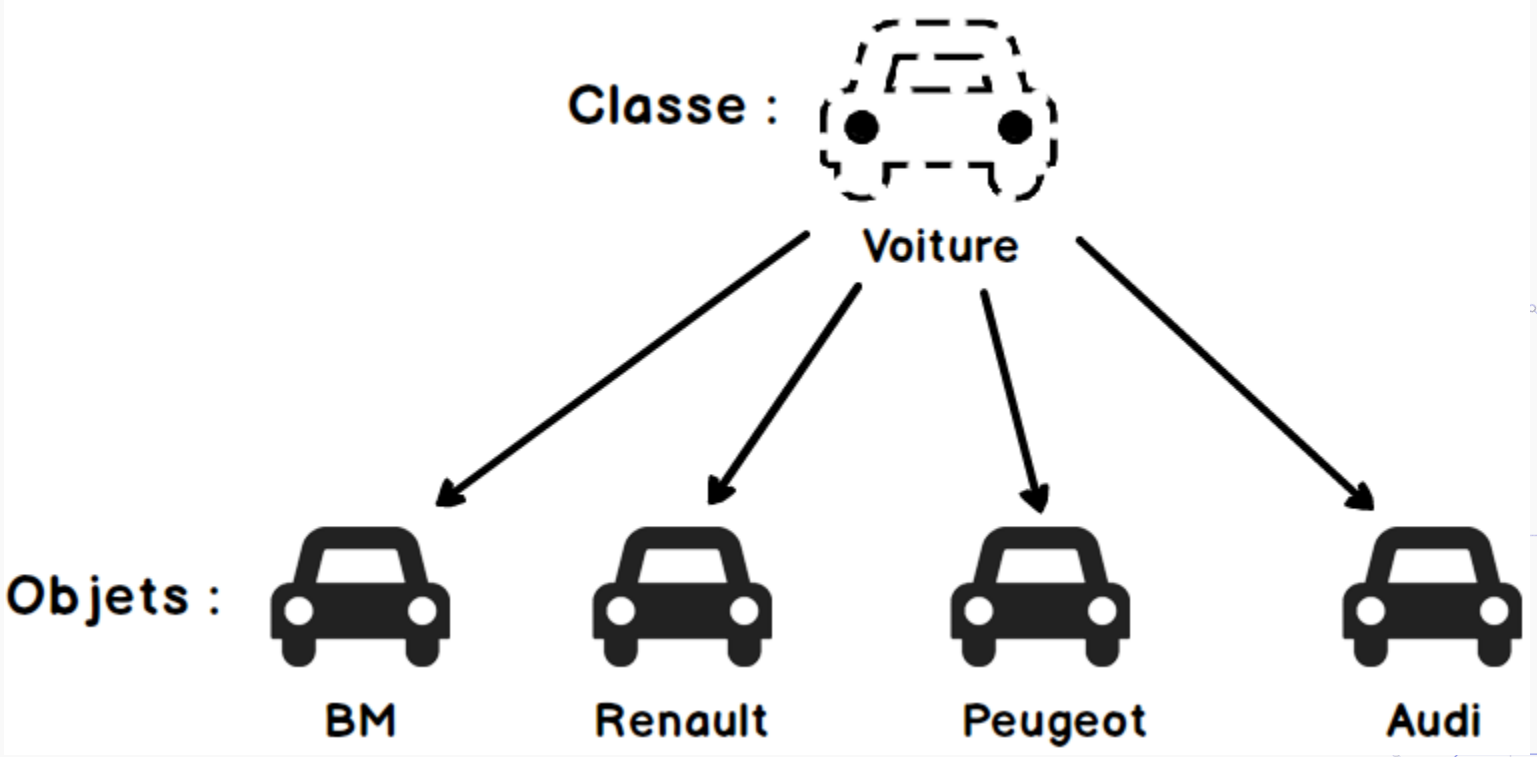
Elle correspond à la déclaration de **l'ensemble des attributs et des méthodes** qui composeront **un objet**.

- **Objet : Une instance d'une classe.**

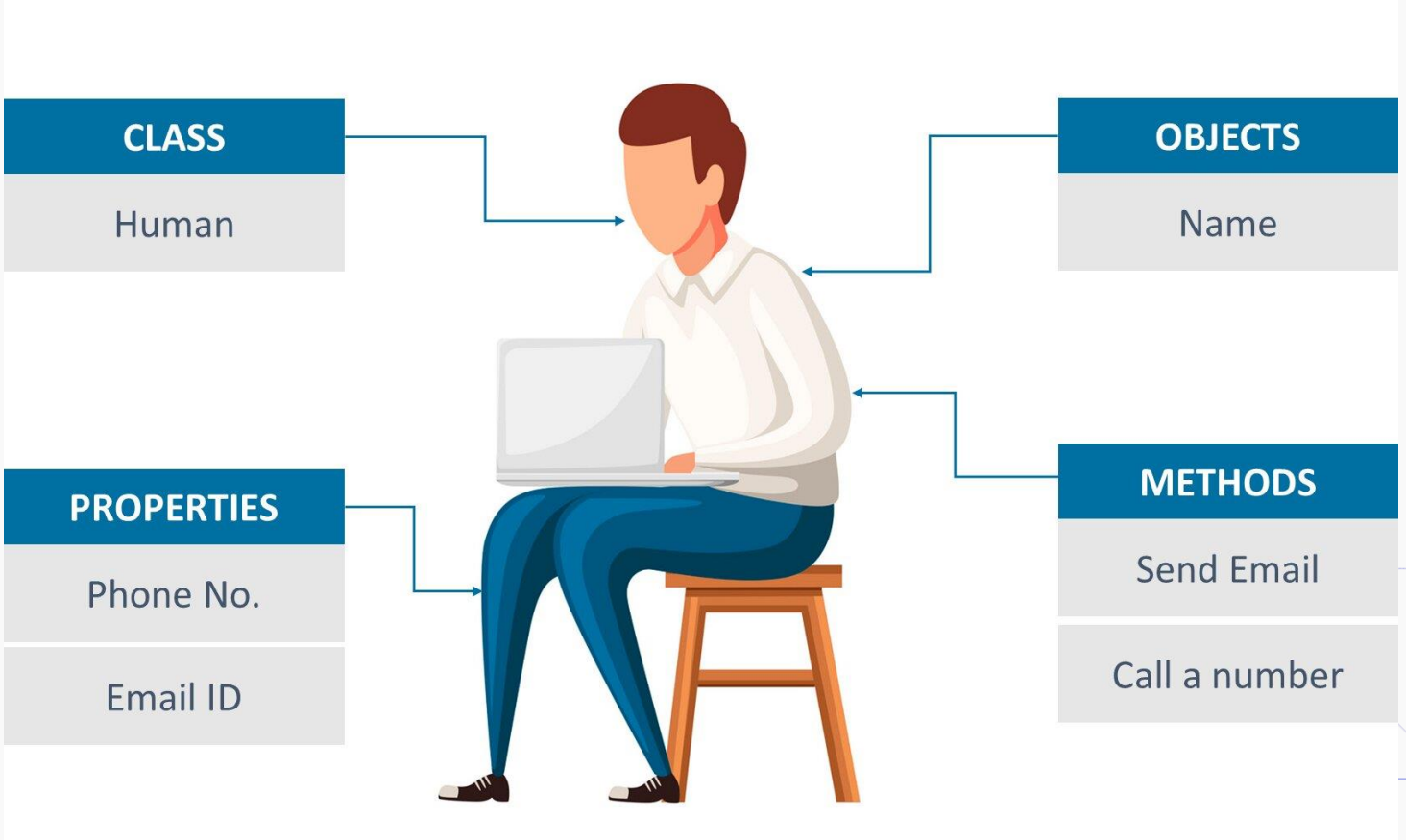
Un objet est créé à partir d'une classe et représente une concrétisation de celle-ci.



Classe et objet



Exemple :


CLASS
Human
OBJECTS
Name
PROPERTIES
Phone No.
Email ID
METHODS
Send Email
Call a number

Classe Vs objet

On appelle classe la structure d'un objet, c'est-à-dire la déclaration de l'ensemble des entités qui composeront un objet.

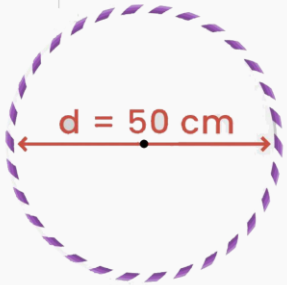
- Un objet est donc « issu » d'une classe, c'est le produit qui sort d'un moule.
- En réalité on dit qu'un objet est une **instanciation** d'une classe

objet = instance

Une classe est composée de deux parties :

- Les **attributs** (parfois appelés *données membres*) : il s'agit des données représentant **l'état de l'objet**
- Les **méthodes** (parfois appelées *fonctions membres*): il s'agit **des opérations applicables aux objets**

Exemple : la classe Cercle



- Un cercle est caractérisé par :
  - Les coordonnées de son centre
  - La valeur de son rayon
- Les traitements à appliquer sur un cercle :
  - Calculer le périmètre
  - Calculer la surface
  - Changer le rayon
  - Déplacer le cercle

Définition de la classe Cercle

Nous allons créer une classe qui contient :

- Deux attributs x et y pour stocker les coordonnées du centre.
- Un attribut rayon pour stocker la taille du cercle.
- Un constructeur pour initialiser ces valeurs.
- Une méthode changer Rayon pour modifier la valeur du rayon.
- Une méthode deplacer pour modifier les coordonnées du centre.

rayon

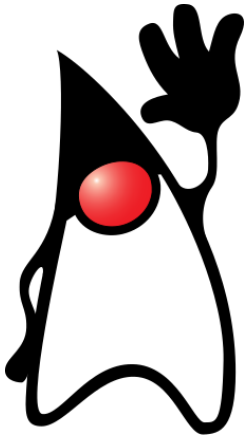
Périmètre du cercle : $P=2\times\pi\times r$

Surface du cercle : $S=\pi\times r2$

π (Pi) = 3,1416

Syntaxe générale d'une classe

```
Package statement
Import statements
modificateur class NomDeLaClasse {
// déclaration des attributs
modificateur type_a1 attribut1 [=valeur];
modificateur type_a2 attribut2;
...
/* déclaration des méthodes */
modific type_m1 nomMethode1(type_p1 param1, ...)
{
    //Corps de methode1
}
...
}
```



Les modificateurs d’accessibilité

•Lors de la conception d’un programme java, le programmeur doit identifier les objets et les données appartenant à chaque objet mais aussi des **droits d’accès** qu’ont les autres objets sur ces données.

•**L’encapsulation de données** dans un objet permet de cacher ou non leur existence aux autres objets du programme pour des raisons de sécurité.
•Une donnée peut être déclarée soit en droit accès : **public, private, protected.**
•On les appelle **les modificateurs d’accessibilité**

Les modificateurs d'accessibilité

Mot-clé	Portée
`public`	Les variables, méthodes ou classes publiques sont accessibles par tout objet.
`private`	Les variables, méthodes ou classes définies comme privées ne sont accessibles que par la classe dans laquelle elles sont définies.
`protected`	Les variables, méthodes ou classes définies comme protégées ne sont accessibles que par les classes filles et classes du même package..
`"rien"`	Idem à public

public – accessible partout

- **Visibilité : Accessible de partout (dans tous les packages).**
- **Utilisation :** Pour rendre les membres ou classes disponibles **pour tous les autres packages.**

```java
package utils;

public class Calculatrice {
    public int additionner(int a, int b) {
        return a + b;
    }
}

// Fichier : Main.java
package autrePackage;

import utils.Calculatrice;

public class Main {
    public static void main(String[] args) {
        Calculatrice calc = new Calculatrice();
        int resultat = calc.additionner(5, 3); // Appel d'une méthode publique
        System.out.println("Résultat : " + resultat); // Affiche : Résultat : 8
    }
}
```

🔒 private

- **<u>Description :</u> accessible seulement dans la même classe**

- **<u>Utilisation :</u>**

Pour masquer les détails d'implémentation et protéger les données sensibles.

Les Attributs et méthodes sont souvent déclarés **private** pour cacher leur implémentation et les protéger contre toute modification externe.

Une **classe** ou **une interface** ne peut pas être déclarée **private.**

❖ Exemple avec **private**

```java
public class CompteBancaire {
    private double solde = 1000;

    public void afficherSolde() {
        System.out.println("Solde : " + solde); // Accès interne possible
    }
}

public class Main {
    public static void main(String[] args) {
        CompteBancaire compte = new CompteBancaire();
        // System.out.println(compte.solde); // Erreur : solde est privé
        compte.afficherSolde(); // Méthode publique pour accéder au solde
    }
}
```

protected

- **Description :** L'élément est accessible dans la même classe, dans **les sous-classes (héritage)** et dans les classes du même package.

- **Utilisation :**

  - Utile pour permettre l'accès aux membres d'une classe parente à ses sous-classes tout en restreignant l'accès aux autres classes extérieures.

❖ Exemple avec **protected**

```
public class Vehicule {
    protected int vitesseMax = 200;
}

public class Voiture extends Vehicule {
    public void afficherVitesseMax() {
        System.out.println("Vitesse max : " + vitesseMax); // Accessible car protected
    }
}

public class Main {
    public static void main(String[] args) {
        Voiture voiture = new Voiture();
        voiture.afficherVitesseMax();
    }
```

Par défaut (aucun modificateur spécifié)

- **Description :** L'élément est accessible uniquement dans le même package.

  C'est ce qu'on appelle l'accès par défaut ou package-private .

- **Utilisation :**

  - Si aucun modificateur n'est spécifié, l'élément est accessible uniquement aux classes du même package.

Exemple

```
class Personne { // Classe par défaut (accessible uniquement dans le même package)
    String nom = "Dupont"; // Attribut par défaut
}

public class Main {
    public static void main(String[] args) {
        Personne personne = new Personne();
        System.out.println(personne.nom); // Accessible car dans le même package
    }
}
```

Quand Utiliser Chaque Modificateur ?

MODIFICATEUR	Usage recommandé
public	Pour les classes, méthodes ou attributs qui doivent être accessibles partout.
private	Pour les attributs et méthodes qui ne doivent être utilisés qu'à l'intérieur de la classe.
protected	Pour les membres qui doivent être accessibles aux sous-classes mais pas au monde extérieur.
Par défaut	Pour les membres qui doivent être partagés uniquement entre les classes d'un même package.

Tableau Récapitulatif des Modificateurs

MODIFICATEUR	CLASSE	PACKAGE	SOUS-CLASSE	MONDE (GLOBAL)
public	✅	✅	✅	✅
protected	✅	✅	✅	❌
Par défaut	✅	✅	❌	❌
private	✅	❌	❌	❌

Règles sur les droits d'accès

Pour respecter **la notion d'encapsulation** des données, il est recommandé de respecter les règles suivantes :

- Toute classe doit avoir un modificateur public
- Tous les attributs d'une classe ont le modificateur **private** afin d'interdire un accès direct à ces attributs.
- Les méthodes sont généralement public sauf si on veut définir une méthode propre uniquement à cette classe.
- **Lorsque un attribut est définit private, l'accès à sa valeur se** fera avec les méthodes : **getter et setter.**

Les getters et setters

Pour la manipulation des attributs *private, on utilise:*

Un **mutateur** (setter): est une méthode qui permet de définir la valeur d'une variable particulière

- Un **accesseur** (getter): est une méthode qui permet d'obtenir la valeur

d'une variable particulière.

- Le setter et le getter doivent être déclarés *public*

```
private float prix;

public void setPrix(float prix){
        this.prix=prix;
}
public float getPrix(){
        return prix;
}
```

```
private boolean absent;

public void setAbsent(boolean absent){
        this.absent=absent;
}
public boolean isAbsent(){
        return absent;
}
```

Les getters et setters

•Une méthode qui a pour but de modifier la valeur d'un attribut d'une classe à la syntaxe :

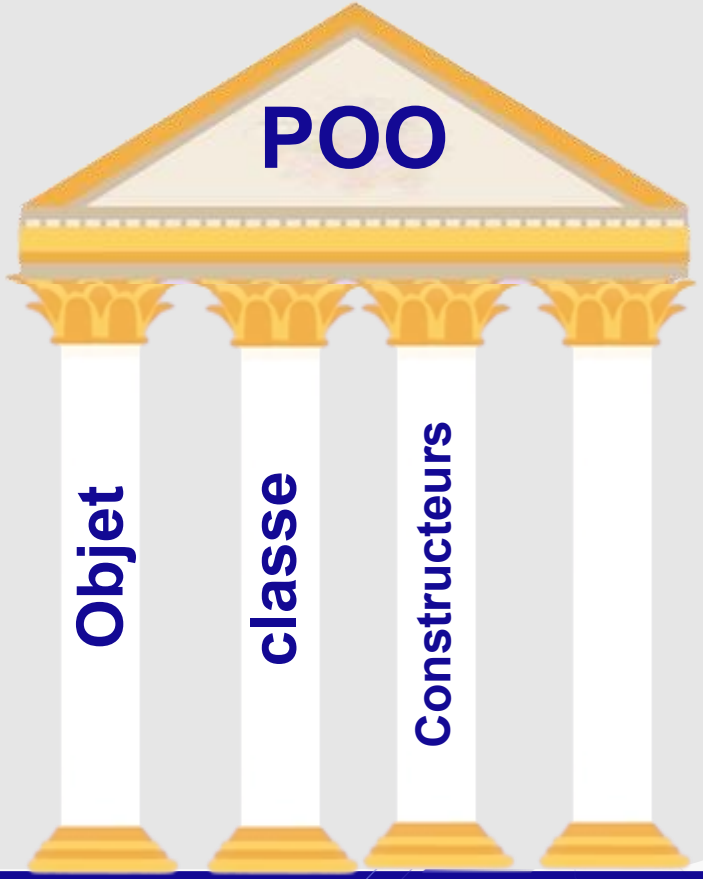
```
public void setNom_attribut(type_p parametre){
nom_attribut = parametre;
}
```

•Une méthodes qui a pour but de récupérer (retourner) la valeur d'un attribut à la syntaxe :

```
public type_attribut getNom_attribut(){
return nom_attribut;
}
```

•Chaque attribut lui sera associé donc une méthode getter et une autre setter quand il est définit en droit d'accès private.

Les constructeurs



Les constructeurs

```java
public class Personne {
        String nom;
        int age;

        public Personne(String nom, int age) {
            super();
            this.nom = nom;
            this.age = age;
        }
```

Un **constructeur** est une méthode spéciale qui est appelée lors de **l'instanciation d'un objet avec new**.
Un constructeur en Java est **une méthode** pour **créer (ou initialiser) un objet d'une classe.**
Il permet **d'initialiser l'objet** avec des valeurs spécifiques.
•Chaque **classe** peut définir **une ou plusieurs méthodes** particulières appelées **constructeurs**.

•**Un constructeur** d'une classe de données est **une méthode** invoquée dans une autre **classe exécutable** pour la création d'un objet de la classe données.
•Cette méthode, qui peut être vide, effectue les opérations nécessaires à l'initialisation d'un objet.

Les constructeurs

Remarques importantes :

Le nom du **constructeur** est **toujours le même que celui de la classe.**

Chaque **constructeur** doit avoir **le même nom que la classe** où il est défini

et **n'a aucune valeur de retour (c'est l'objet créé qui est renvoyé).**

•Plusieurs constructeurs peuvent être définis **s'ils acceptent des paramètres d'entrée différents.**

- Si aucun constructeur n'est défini, Java crée un constructeur par défaut sans argument.

Exemple:

classe

constructeur

```
public class Personne {
        String nom;
        int age;

    public Personne(String nom, int age) {
        super();
        this.nom = nom;
        this.age = age;
    }
```

==> Il **n'a pas de type de retour** (pas de void, ni int, etc.).

Les constructeurs

```java
public class Personne {
        String nom;
        int niveau;

         public Personne() {
        }
```

constructeur

```java
public class MainPersonne {

    public static void main(String[] args) {
        // Créer un objet 'p1' à partir de la classe Personne
        Personne p1 = new Personne();

    }
```

Classe exécutable

Les types de constructeurs :

- **Constructeur par défaut (explicite)**

```
public Personne() {
}
```

- **Constructeur avec paramètres**

```
public Personne(String nom, int niveau) {
    this.nom = nom;
    this.niveau = niveau;
}
```

🧩 À quoi sert un constructeur ?

Il sert à :

- **Créer un objet** à partir d'une classe.
- **Initialiser les variables** (appelées attributs) de cet objet avec des valeurs.

🔑 Caractéristiques d'un constructeur :

- Il **porte exactement le même nom** que la classe.
- Il **n'a pas de type de retour** (pas même void).
- Il peut être **surchargé** (plusieurs constructeurs avec différents paramètres).

🧭 Pourquoi on utilise un constructeur ?

- **Pour donner des valeurs aux objets dès leur création.**
- **Pour éviter d'avoir à appeler plein de méthodes set() après avoir créé l'objet.**
- **Pour personnaliser chaque objet.**

- **Si vous ne créez pas un constructeur dans votre classe, le compilateur va automatiquement vous créer un constructeur par défaut implicite,**

- La plateforme java différencie entre les différents constructeurs déclarés au sein d'une même classe en se basant sur le nombre des paramètres et leurs types.

Les constructeurs

```java
public class Personne {
        int id;
        String nom;
        int niveau;

         public Personne() {
        }

        public Personne(int id) {
            this.id = id;
        }
        public Personne(String nom, int niveau) {
            this.nom = nom;
            this.niveau = niveau;
            }
```

Utilisation :

```java
Personne p1 = new Personne();
```

```java
Personne p2 = new Personne(1235);
```

```java
Personne p3 = new Personne();
    p3.nom = "ahmed";
    p3.niveau = 1;
}
```

🧠 **À retenir :**

- Le constructeur sert à **initialiser un objet**.
- Il s'appelle **automatiquement** lors de la création avec new.
- Il peut avoir **0, 1 ou plusieurs paramètres**.
- Il permet de **gagner du temps** et d'avoir un code **plus propre**.

Exemple :

Constructeur de la classe Cercle

•Ajouter à la classe Cercle, un constructeur avec :

–3 paramètres

–2 paramètres

–1 paramètre (rayon)

–aucun paramètre mais initialise le rayon à 1.

Les objets : Instanciation

•Un objet est une instance d'une classe et est référencé par une variable ayant un état (valeur).

•Pour créer un objet, il est nécessaire de déclarer une variable dont le type est la classe à instancier, puis de faire appel à un constructeur de cette classe en utilisant l'opérateur **new**

Syntaxe
nom_de_la_classe nom_objet;
nom_objet = **new** nom_de_la_classe(…);

Les objets : Instanciation

•On peut simplifier la syntaxe précédente en :

Syntaxe
nom_de_la_classe nom_objet = **new** constructeur_de_la_classe(…);

•Si le constructeur appelé nécessite des paramètres d'entrée, ceux-ci doivent être précisés entre ces **parenthèses** (comme lors d'un appel classique de méthode).

•Exemple : créer plusieurs instances de la classe Cercle avec différents constructeurs.

Les objets : Accès aux attributs et aux méthodes

- Pour accéder à un attribut associée à un objet, il faut préciser l'objet qui le contient.
- Le symbole **.** (point) sert à séparer l'identificateur de l'objet de l'identificateur de la variable
- Syntaxe : **nom_objet.nom_attribut**
- Exemple : accès aux attributs d'un objet produit
- La même syntaxe est utilisée pour appeler une **méthode d'un objet.**
- Syntaxe : **nom_objet.nom_methode();**
- Exemple : accès aux méthodes d'un objet produit

Exécution d’un programme java

- Pour écrire un programme avec le langage java, le programmeur écrit des classes correspondant aux données de son programme.
- Les traitements à effectuer sur les objets de cette classe sont programmés dans les méthodes de celle-ci.
- En général, on définit une autre classe, dite exécutable, dont une méthode appelée **main()** qui manipulera des instances de ces classes appelées **objets**.

La classe exécutable « Main »

```java
public class MainPersonne {

    public static void main(String[] args) {


        Personne p2 = new Personne(1);
        Personne p3 = new Personne("ahmed",1);
        Personne p4 = new Personne(2,"HADHEMI",2);

        p2.afficherInfos();
        p3.afficherInfos();
        p4.afficherInfos();
}
}
```

```java
public class MainPersonne {

    public static void main(String[] args) {


        Personne p2 = new Personne(1);
        Personne p3 = new Personne("ahmed",1);
        Personne p4 = new Personne(2,"HADHEMI",2);

        p2.afficherInfos();
        p3.afficherInfos();
        p4.afficherInfos();
}
}
```

public : Indique que la classe est accessible depuis d'autres classes.

class MainPersonne : Définit une classe appelée MainPersonne.

- Cette classe contient la méthode main, qui est le point d'entrée d'un programme Java.

2. Définition de la méthode main

public : La méthode est accessible par l'extérieur (nécessaire pour le point d'entrée d'un programme en Java).

static : Permet d'exécuter main sans créer d'objet (instance) de la classe MainPersonne.

void : Indique que la méthode ne retourne pas de valeur.

main(String[] args) : main est le nom obligatoire de la méthode principale en Java.

C'est le point d'entrée du programme en Java. Lorsqu'on exécute un programme Java, c'est cette méthode qui est exécutée en premier.

String[] args : C'est un tableau de chaînes de caractères qui permet de passer des arguments au programme lors de son exécution.

Les objets : Accès aux attributs et aux méthodes

- Pour référencer l'objet "courant" (celui dans lequel se situe la ligne de code), le langage Java fournit le mot-clé **this**

- Sa présence est **indispensable** dans le cas où apparait une confusion de notation.

Surcharge des méthodes

- **Une méthode** est une fonction définie dans une classe qui peut être appelée sur un objet.
- Une classe peut contenir des méthodes de même nom, avec des paramètres différents,

→ On dit que la méthode est **surchargée**

```java
package calculatrice;

public class Calculatrice {
    int addition(int a, int b) {
        return a + b;
    }
    int addition(int a, int b, int c) {
        return a + b + c;
    }
}
```

```java
package calculatrice;

public class Main {

    public static void main(String[] args) {
            Calculatrice calc = new Calculatrice();
             System.out.println(calc.addition(5, 10));
             System.out.println(calc.addition(5, 10, 20));
    }

}
```

Attributs d'instance et attributs de classe

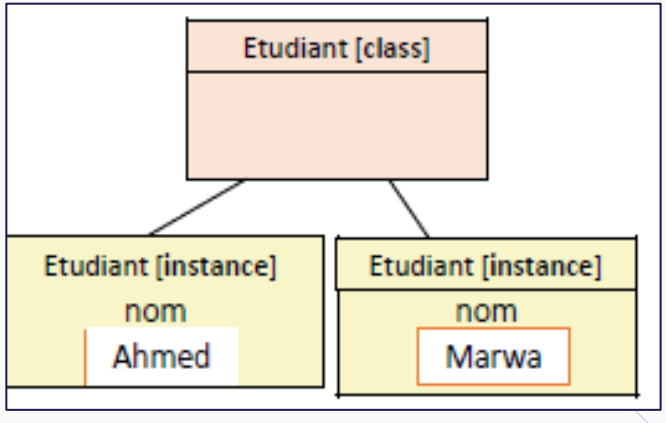
Attributs d'instance

- Chaque instance de la classe possède ses propres valeurs des variables.

```
public class Etudiant{
        public String nom;

        Etudiant(String nom){
        this.nom=nom;}

 }
```




```
Etudiant etud1=new Etudiant ("Ahmed");
Etudiant etud2=new Etudiant ("Marwa");
```

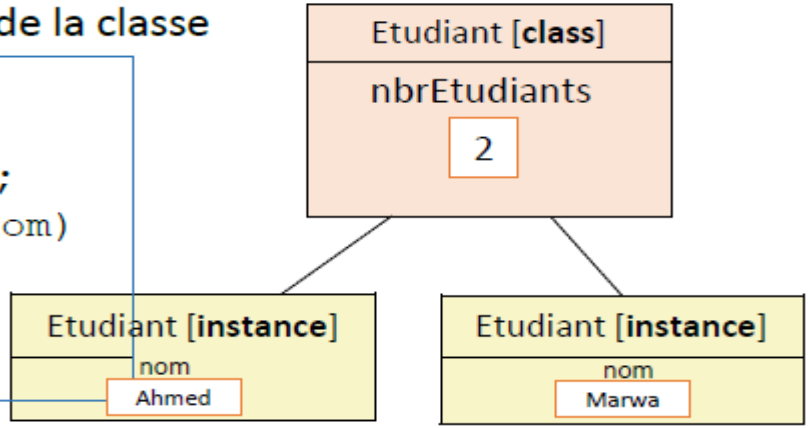
Utilisation: On invoque les attributs d'instance avec le nom de l'instance

```
Class Test{
Public static void main(String[] args){
Etudiant etudiant = new Etudiant();
System.out.println(etudiant.nom); }
}
```

Attributs de classe (static)

- n'appartient pas à une instance particulière, elle appartient à la classe.
- est partagée par toutes les instances de la classe

```
Class Etudiant{
        String nom;
        static int nbrEtudiants;
                Etudiant(String nom)
                { this.nom=nom;
                  nbrEtudiants++;
                }
}
```



Utilisation : On invoque les attibuts static avec le nom de la classe

```
class Test{
    public static void main(String[] args){
  Etudiant etud1 = new Etudiant ("Ahmed");
  Etudiant etud2 = new Etudiant ("Marwa");
      System.out.println(Etudiant.nbrEtudiants);}
}
```

Attributs statiques

- Lors de la création d'une nouvelle instance d’une classe, cet objet possède ces propres attributs afin de pouvoir y stocker des valeurs spécifiques à lui.
- Ainsi pour la **classe Cercle, les attributs x , y et rayon** ont des valeurs différentes d'un objet cercle à un autre.
- Il est parfois utile de spécifier qu'un attribut doit posséder la même valeur pour toutes les instances, autrement dit avoir **une seule valeur partagée par toutes les instances de la classe**.

Attributs statiques

•Si on change la valeur de cet attribut, ce changement sera partagé et connu de toutes les instances.
•Ce sont d'une certaine manière des variables globales au niveau d'une classe.
•Les attributs statiques sont déclarées avec le mot clé static.
C'est pourquoi une telle variable est appelée variable de classe ou variable statique.
•les variables de classe sont préfixées par une référence à un objet ou le nom de la classe (objetClasse1.attribut ou Classe1.attribut).
•Exemple : ajouter un attribut statique indiquant le nombre de cercles crées dans la classe Cercle.

Méthodes statiques

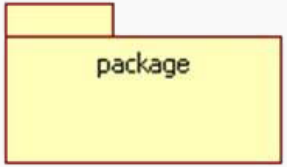
- Une méthode static sera appelée directement avec le nom de la classe ou elle est définit, sans avoir besoin de créer une instance pour faire cet appel.
- Par exemple la fonction main d'une classe exécutable Cl1 sera appelée par la machine virtuelle JVM avec Cl1.main()

Constantes de classe

- Ce sont des cas particuliers de variables de classe, dont les valeurs ne doivent pas être changées.
- On les déclare en combinant les mots clés final avec static.
- Par exemple, dans la classe Math, Math.PI est une constante définie par :

public static final double PI= 3,141592653589;

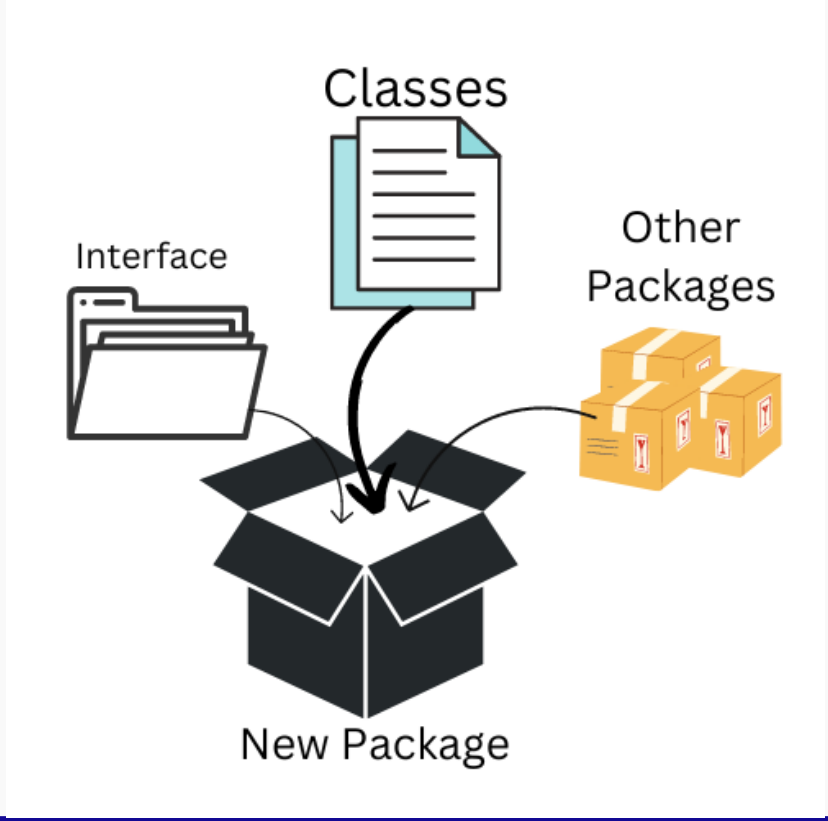
Les packages

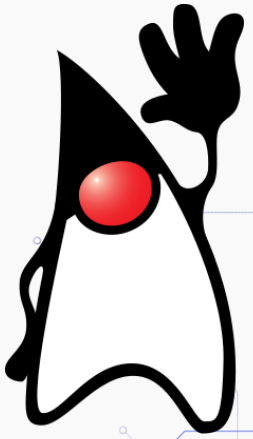


- Un package représente une arborescence indiquant au compilateur quel chemin il faut emprunter pour retrouver la classe.
- Par exemple, si le package est java.util, il va effectuer sa recherche dans le répertoire java/util
- Le mot clé import permet d'importer les classes d'un package, exemples :
  - **import** nomPackage.nomClasse; // Importer une classe d'un package
  - **import** nomPackage.*; //Importer toutes les classes d'un package

Un package peut contenir des sous packages : chaque sous-package est séparé de son parent avec un **point « . »** Il n'y a pas de limite sur le niveau de profondeur.

package com.bts.tp2



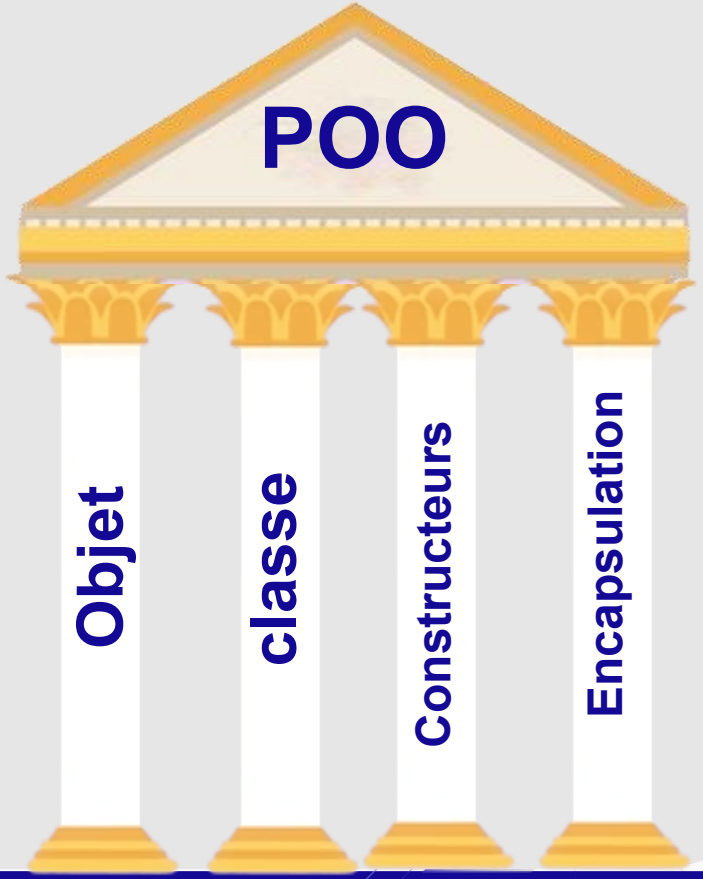


À quoi sert la méthode toString() en Java ?

- La méthode toString() est une méthode définie dans la classe Object, qui est la classe parente de toutes les classes en Java.

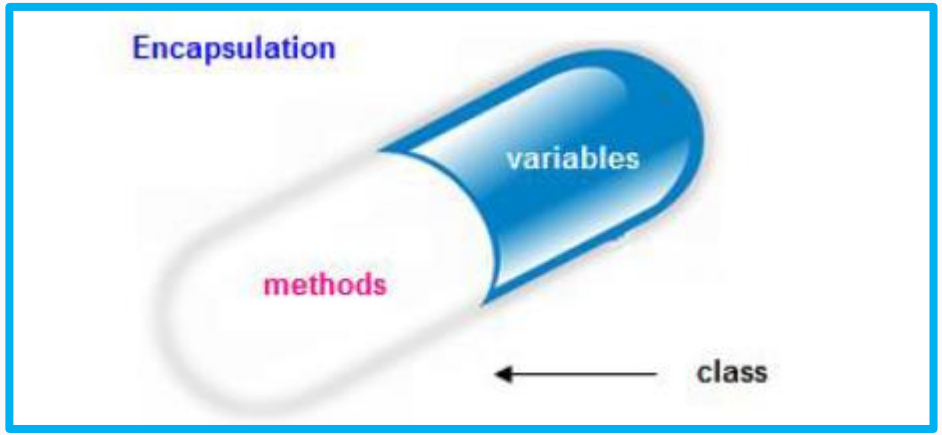
- Elle permet de renvoyer une représentation sous forme de chaîne de caractères (String) d'un objet. Par défaut, si vous n'overridez pas cette méthode dans votre classe, elle retourne une chaîne contenant le nom de la classe suivi du code de hachage de l'objet (par exemple : **Rectangle@1b6d3586).**

L'encapsulation en Java



L'encapsulation

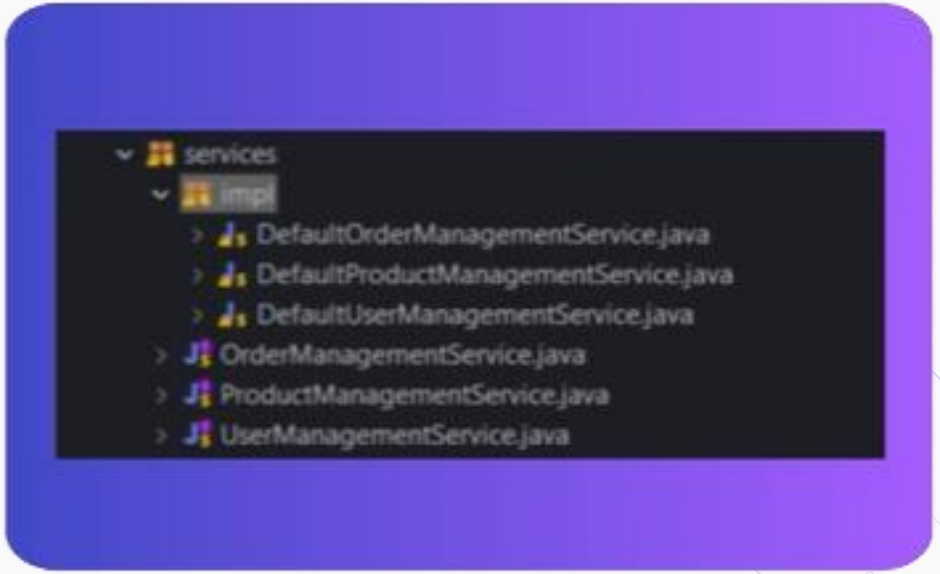
L'encapsulation est **l'un des quatre piliers de la programmation orientée objet (POO)** en Java,
C'est un **principe fondamental** qui consiste à regrouper les **données (attributs)** et les **méthodes** qui les manipulent dans une même classe, tout en **restreignant** l'accès direct aux données pour garantir leur intégrité et leur sécurité.



Encapsulation

L'encapsulation est un principe fondamental de la programmation orientée objet (POO) qui consiste à **cacher les détails internes** d'une classe et à **contrôler l'accès** aux données via des méthodes publiques.

- Private
- Default
- Protected
- Public



🔥 Avantages de l'encapsulation

Avantages	Explication
🛡️ Sécurité	Empêche la modification non contrôlée des attributs.
🎯 Contrôle	Permet de valider les données avant modification.
🏗️ Maintenabilité	Facile à modifier sans impacter le reste du programme.
🧱 Réutilisabilité	Permet d'utiliser les classes dans d'autres projets sans souci.
🔄 Flexibilité	Possibilité de changer l'implémentation interne sans affecter les utilisateurs de la classe.

QUIZ!

Question 1 :

Quelle affirmation est correcte concernant une classe en Java ?

A) Une classe ne peut contenir que des méthodes.

B) Une classe est une instance d'un objet.

C) Une classe est une structure qui définit des attributs et des méthodes.

D) Une classe ne peut pas être instanciée.

Question 2:

Que signifie l'instanciation d’un objet ?

- A) La déclaration d'une variable
- B) La création d'un objet à partir d'une classe
- C) L'héritage d'une classe
- D) La suppression d'une instance

Question 3:

Quel modificateur d’accès permet de restreindre l’accès d’un attribut aux méthodes de la même classe uniquement ?

- A) public
- B) private
- C) protected
- D) default

Explication : Un attribut private ne peut être accessible qu'à l'intérieur de la classe qui le définit

Question 4:

Pourquoi utilise-t-on les getters et setters en Java ?

- A) Pour protéger l'accès direct aux attributs privés
- B) Pour exécuter le programme plus rapidement
- C) Pour éviter la surcharge des méthodes
- D) Pour instancier des objets

Explication : Les getters et setters permettent d'encapsuler les données en restreignant l'accès direct aux attributs tout en offrant des méthodes pour les récupérer ou les modifier.

Question 5:

Quelle est la principale fonction d’un constructeur en Java ?

- a) Exécuter automatiquement un programme
- b) Initialiser un objet lors de sa création
- c) Déclarer une variable statique

Question 6:

Que se passe-t-il si une classe ne définit aucun constructeur ?

a) Java génère automatiquement un constructeur par défaut.

b) La classe ne pourra pas être instanciée.

c) Java affichera une erreur de compilation.

d) La classe sera abstraite par défaut.

Explication : Java génère automatiquement un constructeur par défaut.

Merci pour votre attention